

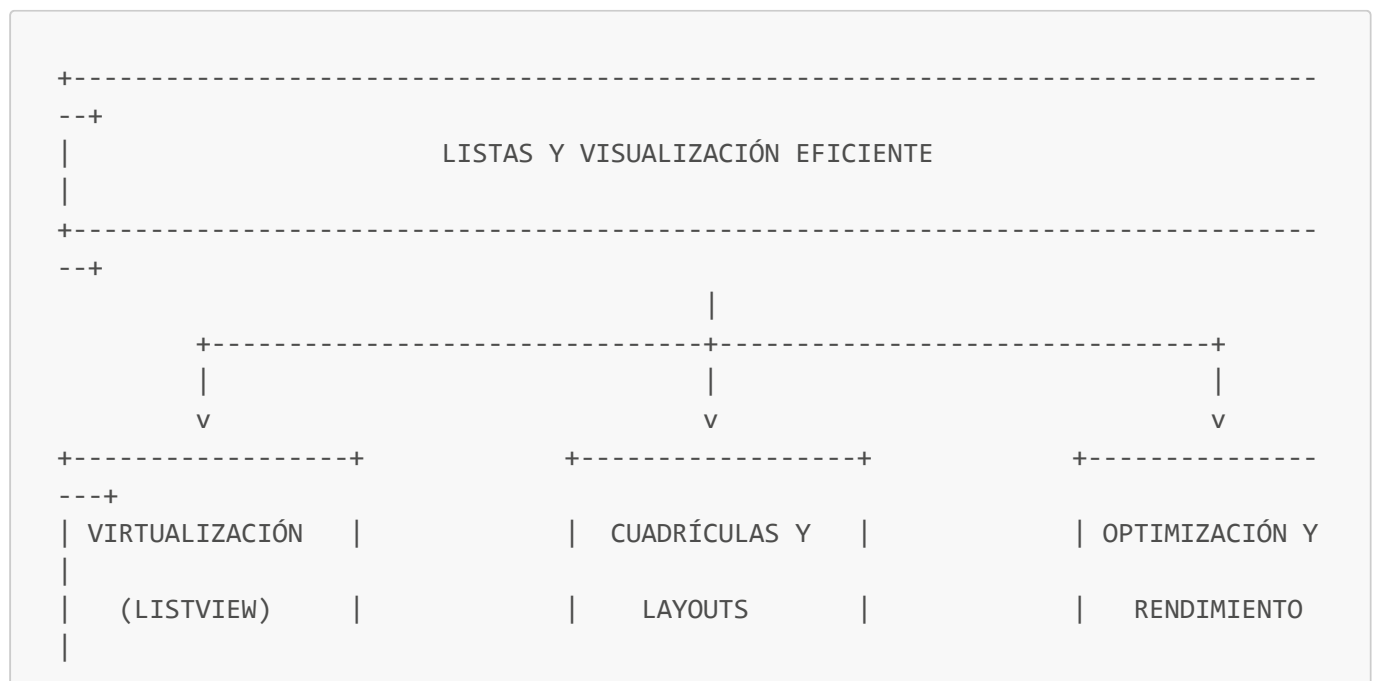
Introducción

Un error grave en la ingeniería de software móvil consiste en intentar renderizar colecciones extensas instanciando un `Column` dentro de un `SingleChildScrollView`. Aunque esta aproximación funciona para formularios breves, resulta catastrófica para el rendimiento en listas grandes: Flutter se ve obligado a instanciar, calcular el layout y pintar **todos y cada uno de los elementos de la lista en memoria al mismo tiempo**, incluso aquellos que se encuentran fuera del área visible del puerto de visión (*Viewport*). Esto provoca caídas drásticas en la tasa de cuadros por segundo (FPS), consumo excesivo de memoria RAM y congelamiento de la interfaz (*Jank*).

Para resolver este desafío, Flutter ofrece widgets de visualización eficiente basados en **desplazamiento diferido o perezoso** (*Lazy Loading / Virtualization*). En esta sesión exploraremos la arquitectura interna de **ListView** y **GridView**, comprendiendo cómo sus constructores optimizados (**.builder** y **.separated**) reciclan elementos en memoria a través del puerto de visión, cómo construir cuadrículas dinámicas adaptativas y qué estrategias aplicar para optimizar el rendimiento de renderizado en listas de gran escala.

Conceptos y Definiciones

Para dominar el renderizado eficiente de colecciones, es necesario diferenciar la instanciación *eagerly* (inmediata) de la virtualización perezosa (*lazy*) dentro de la fase de construcción de Flutter.



	(GRIDVIEW)	(DELEGATES)
<pre> +-----+ ---+ • Default List SliverChild... • ListView.builder • ListView.sepa... AddAutomatic... • ScrollController RepaintBoundary +-----+ ---+ </pre>	<pre> +-----+ • FixedCrossAxis • MaxCrossAxis • AspectRatio • Staggered Grid +-----+ </pre>	<pre> +-----+ • • ItemExtent • • +-----+ </pre>

1. Arquitectura de `ListView` y Reutilización de Memoria

El widget `ListView` es el componente de desplazamiento lineal más utilizado en Flutter. Internamente se apoya en un sistema de *Slivers* que calcula únicamente los elementos que colisionan con la ventana de visualización (*Viewport*).

- **Constructor Predeterminado** `ListView(children: [])`:
- **Comportamiento:** Instancia todos los widgets hijos pasados en la lista `children` de forma inmediata (*eager evaluation*).
- **Uso Recomendado:** Exclusivamente para colecciones pequeñas y fijas (menos de 15 o 20 elementos) donde la complejidad de instanciación es despreciable.
- **Constructor Optimizado** `ListView.builder()`:
- **Comportamiento:** Implementa la virtualización mediante la función `itemBuilder: (context, index)`. Los widgets se instancian únicamente cuando el usuario desplaza la pantalla y el elemento está a punto de entrar en el *Viewport*. Cuando un elemento sale del área de visión más un margen de caché (*cacheExtent*), se destruye o recicla su `Element` y `RenderObject`.
- **Parámetros Clave:**
 - `itemCount`: Define el total de elementos de la lista. Si se omite, la lista se considera infinita.
 - `itemBuilder`: Callback que retorna el widget para una posición `index` específica.
- **Constructor** `ListView.separated()`:
- **Comportamiento:** Variante especializada de `ListView.builder()` que intercala automáticamente un widget separador entre los elementos mediante el callback `separatorBuilder`. Es ideal para dividir registros con líneas horizontales (`Divider`) sin ensuciar la lógica del modelo de datos ni duplicar código dentro del `itemBuilder`.

2. Organización Bidimensional con `GridView`

GridView extiende las capacidades de virtualización de **ListView** al plano bidimensional, organizando los elementos en filas y columnas ordenadas.

- **Estrategias de Delegación de Cuadrícula (**gridDelegate**):**
 - **SliverGridDelegateWithFixedCrossAxisCount:** Define un número **fijo de columnas** en el eje cruzado (por ejemplo, exactamente 2 o 3 columnas sin importar el ancho de la pantalla).
 - **SliverGridDelegateWithMaxCrossAxisExtent:** Define un **ancho máximo por celda** (por ejemplo, máximo 150 píxeles por tarjeta). Flutter calcula automáticamente el número de columnas dependiendo del ancho de la pantalla, lo que garantiza una interfaz receptiva (*responsive*) entre móviles y tablets.
 - **childAspectRatio:** Propiedad crítica en ambos delegados que define la relación entre el ancho y el alto de cada celda (**Ancho/Alto**). Una relación de **1.0** genera celdas totalmente cuadradas.
-

3. Técnicas de Optimización de Memoria y Rendimiento

Para garantizar despliegues fluidos a 60 FPS (o 120 FPS en pantallas de alta frecuencia de refresco), debemos aplicar las siguientes optimizaciones en listas extensas:

- **Uso de **itemExtent** o **prototypeItem**:** Si todos los elementos de un **ListView** tienen una altura (o ancho) fija conocida, declarar la propiedad **itemExtent** permite al motor de Flutter omitir la fase de cálculo de dimensiones (*Layout Phase*) de cada hijo durante el desplazamiento, calculando la posición absoluta de los elementos mediante aritmética directa ($Y = index \times itemExtent$).
 - **Gestión de Flags de Retención:**
 - **addAutomaticKeepAlives: true:** Mantiene activo el estado (**State**) de los elementos fuera de pantalla (útil para campos de texto dentro de listas), aunque incrementa el uso de memoria RAM.
 - **addRepaintBoundaries: true:** Aísla las operaciones de pintado de cada elemento en su propia capa, evitando que el rediseño de un elemento fuerce el repintado de sus vecinos.
 - **Control de Desplazamiento (**ScrollController**):** Permite escuchar los eventos de desplazamiento en tiempo real (**addListener**), detectar cuando el usuario alcanza el final de la lista para implementar la técnica de **Desplazamiento Infinito / Carga Perezosa** (*Pagination / Infinite Scroll*), o realizar desplazamientos programáticos mediante **animateTo()**.
-

Ejemplo Completo con Código Fuente: Monitor de Registros e Inventario de Servidores

A continuación se presenta una implementación integral que combina un **ListView.separated** virtualizado con paginación infinita, un **GridView.builder** adaptable con **SliverGridDelegateWithMaxCrossAxisExtent**, un **ScrollController** con botón de retorno al inicio y un selector de modo de visualización.

Código Fuente Explicado (**lib/main.dart**)

```
import 'package:flutter/material.dart';

void main() {
  runApp(const DemoListasEficientesApp());
}

/// Widget Raíz de la Aplicación
class DemoListasEficientesApp extends StatelessWidget {
  const DemoListasEficientesApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Visualización Eficiente',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF003366), // Azul Institucional
        ),
      ),
      home: const PantallaMonitorRegistros(),
    );
  }
}

/// Modelo de datos inmutable para los registros de red
class RegistroServidor {
  final String id;
  final String servidor;
  final String direccionIP;
  final String estado;
  final int latenciaMs;
  final DateTime fechaRegistro;

  RegistroServidor({
    required this.id,
    required this.servidor,
    required this.direccionIP,
    required this.estado,
    required this.latenciaMs,
    required this.fechaRegistro,
  });
}

/// Pantalla Principal con Visualización Eficiente de Colecciones
class PantallaMonitorRegistros extends StatefulWidget {
  const PantallaMonitorRegistros({super.key});

  @override
  State<PantallaMonitorRegistros> createState() =>
    _PantallaMonitorRegistrosState();
}
```

```

}

class _PantallaMonitorRegistrosState extends State<PantallaMonitorRegistros> {
  // Controlador de desplazamiento para paginación y eventos
  final ScrollController _scrollController = ScrollController();

  // Colección dinámicamente extendida (Simulación de Dataset de miles de
  elementos)
  final List<RegistroServidor> _registros = [];

  bool _modoCuadrícula = false;
  bool _cargandoMasDatos = false;
  bool _mostrarBotonTop = false;

  @override
  void initState() {
    super.initState();
    _generarRegistrosIniciales(50);

    // Escuchar el desplazamiento para detectar el final de la lista y mostrar el
    botón Top
    _scrollController.addListener(() {
      // Mostrar u ocultar botón de retorno según la posición del scroll
      if (_scrollController.offset > 300 && !_mostrarBotonTop) {
        setState(() => _mostrarBotonTop = true);
      } else if (_scrollController.offset <= 300 && _mostrarBotonTop) {
        setState(() => _mostrarBotonTop = false);
      }
    });

    // Detectar si el usuario está cerca del final para cargar más datos
    (Infinite Scroll)
    if (_scrollController.position.pixels >=
        _scrollController.position.maxScrollExtent - 200 &&
        !_cargandoMasDatos) {
      _cargarMasRegistros();
    }
  });
}

@override
void dispose() {
  _scrollController.dispose();
  super.dispose();
}

/// Generación simulación de datos en memoria
void _generarRegistrosIniciales(int cantidad) {
  final baseId = _registros.length;
  for (int i = 1; i <= cantidad; i++) {
    final idActual = baseId + i;
    _registros.add(
      RegistroServidor(
        id: 'LOG-${idActual.toString().padLeft(5, '0')}',
        servidor: 'Node-Cluster-${(idActual % 8) + 1}',
      ),
    );
  }
}

```

```

        direccionIP: '10.0.${(idActual % 255)}.${(idActual % 100) + 1}',
        estado: (idActual % 7 == 0) ? 'ERROR' : (idActual % 5 == 0 ? 'WARN' :
'OK'),
        latenciaMs: 12 + (idActual % 150),
        fechaRegistro: DateTime.now().subtract(Duration(minutes: idActual * 2)),
      ),
    );
  }
}

/// Carga perezosa de más registros al alcanzar el final
Future<void> _cargarMasRegistros() async {
  setState(() => _cargandoMasDatos = true);

  // Simulación de latencia de red de 1.2 segundos
  await Future.delayed(const Duration(milliseconds: 1200));

  setState(() {
    _generarRegistrosIniciales(30);
    _cargandoMasDatos = false;
  });
}

/// Retorno suave a la parte superior de la lista
void _irAlInicio() {
  _scrollController.animateTo(
    0,
    duration: const Duration(milliseconds: 600),
    curve: Curves.easeInOut,
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Telemetría de Red (${_registros.length} items)'),
      backgroundColor: Theme.of(context).colorScheme.primary,
      foregroundColor: Colors.white,
      actions: [
        IconButton(
          icon: Icon(_modoCuadrícula ? Icons.view_list : Icons.grid_view),
          tooltip: _modoCuadrícula ? 'Ver como Lista' : 'Ver como Cuadrícula',
          onPressed: () {
            setState(() {
              _modoCuadrícula = !_modoCuadrícula;
            });
          },
        ),
      ],
    ),
    body: Column(
      children: [
        // Banner de Métricas Rápidas

```

```

Container(
  padding: const EdgeInsets.symmetric(horizontal: 16.0, vertical: 8.0),
  color: Colors.blue.shade50,
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
      Text(
        'Modo: ${_modoCuadricula ? "GridView.builder" :
"ListView.separated"}',
        style: const TextStyle(fontWeight: FontWeight.bold, fontSize:
12.0),
      ),
      const Text(
        'Virtualización: LAZY (Perezosa)',
        style: TextStyle(color: Colors.green, fontWeight:
FontWeight.bold, fontSize: 12.0),
      ),
    ],
  ),
),

// VISTA PRINCIPAL: LISTA O CUADRÍCULA VIRTUALIZADA
Expanded(
  child: _modoCuadricula
    ? _buildVistaCuadricula()
    : _buildVistaListaSeparada(),
),

// Indicador de Carga al Paginando
if (_cargandoMasDatos)
  Container(
    padding: const EdgeInsets.all(8.0),
    color: Colors.black12,
    child: Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: const [
        SizedBox(
          width: 16,
          height: 16,
          child: CircularProgressIndicator(strokeWidth: 2),
        ),
        SizedBox(width: 10),
        Text('Cargando más registros perezosamente...', style:
TextStyle(fontSize: 12)),
      ],
    ),
  ),

// Botón flotante para retorno rápido a la parte superior
floatingActionButton: _mostrarBotonTop
  ? FloatingActionButton.small(
    onPressed: _irAlInicio,

```

```

        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
        child: const Icon(Icons.arrow_upward),
      )
      : null,
    );
  }

// =====
// CONSTRUCCIÓN DE LISTA MEDIANTE ListView.separated
// =====
Widget _buildVistaListaSeparada() {
  return ListView.separated(
    controller: _scrollController,
    // OPTIMIZACIÓN: Si la altura de cada ListTile es fija, itemExtent acelera
    el layout
    itemExtent: 72.0,
    itemCount: _registros.length,
    separatorBuilder: (context, index) => const Divider(height: 1, indent: 16,
endIndent: 16),
    itemBuilder: (context, index) {
      final registro = _registros[index];
      return ListTile(
        key: ValueKey(registro.id),
        leading: CircleAvatar(
          backgroundColor:
_gettenerColorEstado(registro.estado).withOpacity(0.2),
          child: Icon(
            _obtenerIconoEstado(registro.estado),
            color: _obtenerColorEstado(registro.estado),
            size: 20,
          ),
        ),
        title: Text(
          '${registro.id} - ${registro.servidor}',
          style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 14),
        ),
        subtitle: Text(
          'IP: ${registro.direccionIP} | Latencia: ${registro.latenciaMs}ms',
          style: const TextStyle(fontSize: 12),
        ),
        trailing: Text(
          '${registro.fechaRegistro.minute}m atrás',
          style: const TextStyle(color: Colors.grey, fontSize: 10),
        ),
      );
    },
  );
}

// =====
// CONSTRUCCIÓN DE CUADRÍCULA MEDIANTE GridView.builder
// =====
Widget _buildVistaCuadricula() {

```

```

return GridView.builder(
  controller: _scrollController,
  padding: const EdgeInsets.all(12.0),
  // DELEGADO RECEPTIVO: Calcula columnas dinámicamente según el ancho
  gridDelegate: const SliverGridDelegateWithMaxCrossAxisExtent(
    maxCrossAxisExtent: 180.0, // Ancho máximo por celda
    childAspectRatio: 1.1,      // Relación Ancho / Alto
    crossAxisSpacing: 10.0,
    mainAxisSpacing: 10.0,
  ),
  itemCount: _registros.length,
  itemBuilder: (context, index) {
    final registro = _registros[index];
    final color = _obtenerColorEstado(registro.estado);

    return Card(
      key: ValueKey(registro.id),
      elevation: 2,
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(10)),
      child: Padding(
        padding: const EdgeInsets.all(8.0),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Icon(_obtenerIconoEstado(registro.estado), color: color, size:
28),

            const SizedBox(height: 6),
            Text(
              registro.id,
              style: const TextStyle(fontWeight: FontWeight.bold, fontSize:
12),
            ),
            Text(
              registro.direccionIP,
              style: const TextStyle(color: Colors.grey, fontSize: 10),
            ),
            const SizedBox(height: 6),
            Container(
              padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 2),
              decoration: BoxDecoration(
                color: color.withOpacity(0.15),
                borderRadius: BorderRadius.circular(4),
              ),
              child: Text(
                '${registro.latenciaMs} ms',
                style: TextStyle(color: color, fontWeight: FontWeight.bold,
fontSize: 10),
              ),
            ),
          ],
        ),
      ),
    );
  },
);

```

```
    );  
  }  
  
  // Métodos auxiliares de presentación  
  Color _obtenerColorEstado(String estado) {  
    switch (estado) {  
      case 'OK':  
        return Colors.green.shade700;  
      case 'WARN':  
        return Colors.orange.shade800;  
      case 'ERROR':  
        return Colors.red.shade700;  
      default:  
        return Colors.grey;  
    }  
  }  
  
  IconData _obtenerIconoEstado(String estado) {  
    switch (estado) {  
      case 'OK':  
        return Icons.check_circle;  
      case 'WARN':  
        return Icons.warning;  
      case 'ERROR':  
        return Icons.error;  
      default:  
        return Icons.dns;  
    }  
  }  
}
```

Prácticas Guiadas / Laboratorio

Práctica 1: Impacto del Rendimiento entre `SingleChildScrollView + Column` y `ListView.builder`

Esta práctica demuestra experimentalmente la diferencia en consumo de recursos y tiempo de construcción entre una lista no virtualizada y una virtualizada.

1. Abre el archivo principal de tu proyecto y crea una lista de datos de 5,000 elementos de texto simple.
2. Construye una vista alternativa que renderice los 5,000 elementos utilizando la siguiente estructura no recomendada:

```
SingleChildScrollView(  
  child: Column(  
    children: datos.map((e) => Text(e)).toList(),  
  ),  
)
```

)

3. Ejecuta la aplicación en modo de depuración y abre las **Flutter DevTools** (pestaña *Performance*).
4. Mide el pico de uso de memoria RAM y observa la presencia de congelamientos (*Jank frames*) al cargar la pantalla.
5. Reemplaza la vista por un `ListView.builder()` con `itemCount: datos.length`.
6. **Conclusión y Análisis:** Observa en DevTools cómo con `ListView.builder()` el consumo de memoria se mantiene plano y constante, dado que el número de instancias de widgets creadas equivale estrictamente a las que caben visualmente en la pantalla (aproximadamente 12 a 15 elementos a la vez).

Práctica 2: Implementación de Paginación Infinita (*Infinite Scroll*) con Indicador Visual

1. Utilizando el código de la sesión, modifica el listener del `ScrollController` para que la carga de datos se dispare cuando la posición del scroll alcance el 85% del desplazamiento total

```
(_scrollController.position.pixels >= _scrollController.position.maxScrollExtent * 0.85).
```
2. Agrega una verificación de seguridad para evitar que se ejecuten peticiones duplicadas simultáneas mientras la bandera `_cargandoMasDatos` esté en `true`.
3. Añade un elemento final al `ListView.separated` agregando un incremento de +1 al `itemCount` para renderizar un widget `CircularProgressIndicator` directamente como el último elemento de la lista.

Resumen y Puntos Clave

En esta sesión hemos estudiado las herramientas fundamentales para la representación visual eficiente de grandes volúmenes de datos en Flutter:

- **Peligro de `SingleChildScrollView + Column`:** Crea e instancia todos los hijos en memoria simultáneamente, siendo inviable para colecciones grandes o dinámicas.
- **Virtualización con `ListView.builder` y `ListView.separated`:** Instancian y renderizan únicamente los elementos visibles en el *Viewport*, destruyendo o reciclando los elementos fuera de pantalla para mantener bajo el consumo de memoria.
- **Flexibilidad con `GridView.builder`:** Permite organizar datos en dos dimensiones. Con `SliverGridDelegateWithMaxCrossAxisExtent` se logra una maquetación adaptativa según el ancho del dispositivo.
- **Optimización Extrema mediante `itemExtent`:** Permite omitir el cálculo de layout de los hijos si todos tienen una dimensión fija en el eje de desplazamiento.
- **Control de Navegación con `ScrollController`:** Permite escuchar la posición actual del desplazamiento, habilitando técnicas como paginación infinita (*Infinite Scroll*) y animaciones de retorno al inicio.

Bibliografía y Referencias

1. **Google LLC.** (2024). *Displaying data efficiently with ListViews and GridViews in Flutter*. Recuperado de: <https://docs.flutter.dev/ui/widgets/layout#listview>
2. **Flutter Engineering Team.** (2023). *Scrolling mechanics and Slivers in Flutter*. Recuperado de: <https://docs.flutter.dev/ui/advanced/slivers>
3. **Smit, A.** (2022). *Flutter Performance Optimization: Handling Large Datasets and Scrolling Mechanics*. Packt Publishing. ISBN: 978-1803234908.
4. **Windmill, E.** (2020). *Flutter in Action* (Capítulo 6: Scrollable Widgets and Lazy Loading). Manning Publications. ISBN: 978-1617296147.
5. **Apress.** (2023). *Advanced Mobile Application Architecture in Dart and Flutter*. ISBN: 978-1484291810.